DEEP ARCHITECTURAL GUIDE & DEFENSE MASTERCLASS

V-JEPA 2.1 vs. Qwen2.5-VL for Video Action Recognition

Prepared for Master of Science in Information Technology (M.Sc. IT / MIT) Thesis Defense

Central Department of Computer Science & Information Technology (CDCSIT), IoST, Tribhuvan University, Nepal

---

Thesis Title: *Enhancing Self-Supervised Video Representations with HAF-STD Net: A Comparative Evaluation Against VLM for Action Recognition*

Candidate: Erize Rai | **Roll No:** 8025006 | **Reg. No:** 5-3-28-193-2023

Supervisor: Asst. Prof. Dr. Dhiraj Kedar Pandey

Contents

1 Executive Overview: Why This Matters for Your Defense 3

2 Architecture 1: V-JEPA 2.1 (Meta AI) 3

2.1 Foundational Concept: The JEPA Paradigm . . . 3

2.2 Step-by-Step Architectural Pipeline of V-JEPA 2.1 . . . 4

2.2.1 1. Spatiotemporal Tokenization (3D Tubelet Convolution) . . . 4

2.2.2 2. 3D Rotary Position Embeddings (3D-RoPE) . . . 4

2.2.3 3. Context Encoder E_θ (x-Encoder) . . . 4

2.2.4 4. Target Encoder $E_{\bar{\theta}}$ (y-Encoder) & Exponential Moving Average (EMA) . . . 5

2.2.5 5. Predictor Network P_ϕ . . . 5

2.2.6 6. Innovations in V-JEPA 2.1 vs. Previous V-JEPA Versions . . . 5

2.2.7 7. Multi-Layer Feature Extraction for HAF-STD Net . . . 6

3 Architecture 2: Qwen2.5-VL (Alibaba Cloud) 6

3.1 Foundational Concept: The Multimodal Autoregressive VLM . . . 6

3.2 Step-by-Step Architectural Pipeline of Qwen2.5-VL . . . 7

3.2.1 1. Native Dynamic Resolution Vision Encoder . . . 7

3.2.2 2. Window Attention Mechanism . . . 7

3.2.3 3. Multimodal Rotary Position Embedding (MRoPE) . . . 7

3.2.4 4. Vision-to-Language Merger (MLP Projector) . . . 8

3.2.5 5. Autoregressive Language Model Decoder . . . 8

3.2.6 6. 4-bit NormalFloat (NF4) Quantization . . . 8

3.2.7 7. Feature Extraction Protocol . . . 8

4 Comprehensive Architecture Comparison Matrix 9

5 Why V-JEPA 2.1 Beat Qwen2.5-VL: The Defense Argument 9

6 Viva Defense Practice: Tough Examiner Questions & Model Answers 10

7 Defense Quick-Recall Cheat Sheets **15**

1 Executive Overview: Why This Matters for Your Defense

In your thesis defense, the committee will scrutinize the foundational premises of your comparative study. Your research juxtaposes two fundamentally distinct paradigms in modern visual representation learning:

1. **V-JEPA 2.1 (Meta AI, 654M parameters):** A non-generative, self-supervised video foundation model trained entirely in *latent representation space* through masked spatiotemporal prediction without text guidance or pixel reconstruction.
2. **Qwen2.5-VL (Alibaba Cloud, 7B parameters):** A multimodal autoregressive vision-language model trained jointly on massive image-text-video corpora, equipped with dynamic-resolution vision encoding and causal language decoding.

The Core Defense Thesis Argument

Your key empirical finding is that **V-JEPA 2.1 paired with HAF-STD Net achieves 98.1% accuracy (97.8% macro F1) on FineGym and 84.7% accuracy (85.3% macro F1) on HMDB51**, outperforming Qwen2.5-VL (88.4% F1 on FineGym, 79.1% F1 on HMDB51) by substantial margins (+9.4% and +6.2% F1), despite V-JEPA 2.1 having $\sim$**10$\times$ fewer parameters** (654M vs. 7B).
Examiners will ask: "Why does a model with 10$\times$ fewer parameters beat an advanced 7B vision-language foundation model?" This document equips you with the exact architectural, mathematical, and representational answers to dominate that question.

2 Architecture 1: V-JEPA 2.1 (Meta AI)

2.1 Foundational Concept: The JEPA Paradigm

Traditional self-supervised vision methods rely on one of two paradigms:

- **Generative / Pixel-level Reconstruction (MAE, VideoMAE):** Predict missing pixels $x \in \mathbb{R}^3$. *Flaw:* Forces the network to waste capacity modeling high-frequency unpredictable noise (foliage texture, lighting fluctuations, background static) rather than high-level semantic dynamics.
- **Contrastive Learning (SimCLR, MoCo):** Push augmented views together while pushing negative samples apart. *Flaw:* Requires delicate negative sample mining, large batch sizes, and heavy handcrafted data augmentations that can alter physical motion semantics.

Joint Embedding Predictive Architecture (JEPA), proposed by Yann LeCun, eliminates both drawbacks: it predicts the *representation of masked regions in a learned latent feature space*. Because the loss is calculated in feature space, the encoder is free to discard low-level pixel noise and retain only semantic spatiotemporal structure.

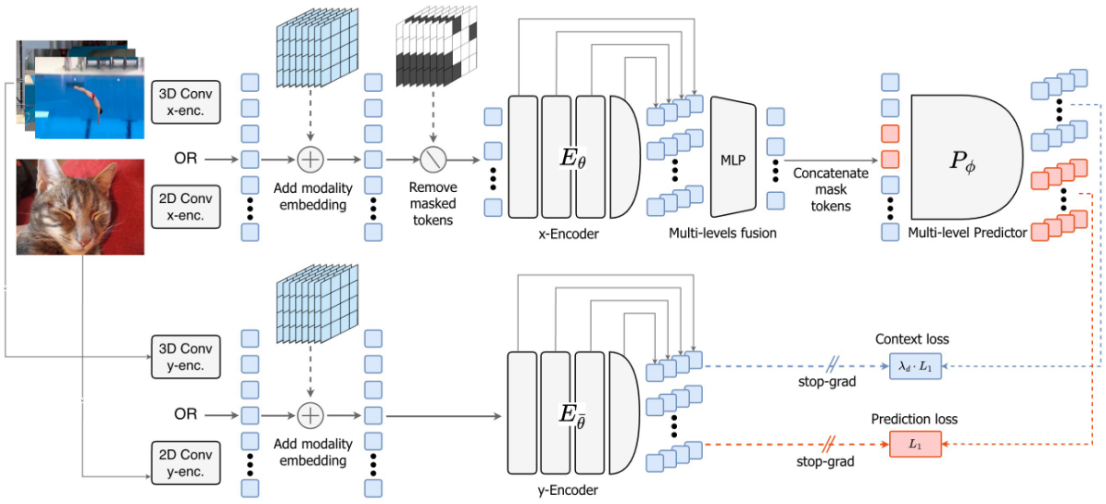


Figure 1: Architecture of V-JEPA 2.1 self-supervised pretraining framework: dual x-Encoder and y-Encoder pipeline with multi-level intermediate fusion, mask-denoising predictor, and stop-gradient EMA targets.

2.2 Step-by-Step Architectural Pipeline of V-JEPA 2.1

2.2.1 1. Spatiotemporal Tokenization (3D Tubelet Convolution)

Given an input video clip $\mathbf{V} \in \mathbb{R}^{B\times T\times H\times W\times C}$ ($T = 8$ frames, $H = W = 224$, $C = 3$ channels):

- Non-overlapping spatiotemporal *tubelets* of size $2\times16\times16$ (temporal $\times$ height $\times$ width) are extracted using a 3D convolution layer with kernel size $(2, 16, 16)$ and stride $(2, 16, 16)$.
- Temporal token count: $T' = \frac{T}{2} = \frac{8}{2} = 4$.
- Spatial token grid: $H' = \frac{224}{16} = 14$, $W' = \frac{224}{16} = 14$.
- Total tokens per clip: $N = T' \times H' \times W' = 4 \times 14 \times 14 = 784$ tokens.
- Each tubelet is projected into hidden dimension $D = 1280$ (Vision Transformer Huge / ViT-H):

$$\mathbf{z}_0 = \text{Conv3D}(\mathbf{V}) + \mathbf{E}_{\text{3D-RoPE}}, \quad \mathbf{z}_0 \in \mathbb{R}^{B\times784\times1280} \tag{1}$$

2.2.2 2. 3D Rotary Position Embeddings (3D-RoPE)

Unlike standard sinusoidal 1D position encodings that treat video as an unrolled sequence, V-JEPA 2.1 injects 3D coordinates (t, h, w) directly into query-key attention matrices using rotary transformations:

$$\mathbf{q}_m^T\mathbf{k}_n = (\mathbf{R}^{3D}_{\Theta,m}\mathbf{W}_q\mathbf{x}_m)^T(\mathbf{R}^{3D}_{\Theta,n}\mathbf{W}_k\mathbf{x}_n) = \mathbf{x}_m^T\mathbf{W}_q^T\mathbf{R}^{3D}_{\Theta,n-m}\mathbf{W}_k\mathbf{x}_n \tag{2}$$

This preserves exact relative spatiotemporal distances across both spatial planes and temporal trajectories.

2.2.3 3. Context Encoder E_θ (x-Encoder)

- **Backbone:** Vision Transformer Huge (ViT-H/16).
- **Parameters:** 654 Million.
- **Layers:** $L = 32$ Transformer blocks.
- **Attention Heads:** 16 heads, each of dimension $d_k = \frac{1280}{16} = 80$.
- **FFN Hidden Dimension:** $4 \times D = 5120$.
- **Precision:** Float16 with Scaled Dot-Product Attention (SDPA / FlashAttention).

- **Operation:** The context encoder processes the visible, unmasked spatiotemporal patches:

$$\mathbf{z}'_l = \text{MHSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \quad \mathbf{z}_l = \text{FFN}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l \tag{3}$$

2.2.4 4. Target Encoder $E_{\bar{\theta}}$ (y-Encoder) & Exponential Moving Average (EMA)

To supply target representations without collapsing to trivial constant representations:

- The target encoder $E_{\bar{\theta}}$ shares the exact same ViT-H architecture as E_θ.
- It processes the complete video view y.
- **CRITICAL DEFENSE POINT:** The target encoder receives **NO gradient descent updates** ($\nabla_{\bar{\theta}}\mathcal{L} = 0$). Its weights are updated strictly via Exponential Moving Average (EMA):

$$\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta, \quad \tau \in [0.996, 1.0) \tag{4}$$

  During pretraining, τ follows a cosine schedule starting at 0.996 and ramping up to 1.0.

2.2.5 5. Predictor Network P_ϕ

The predictor is a lightweight Vision Transformer (typically 12 layers, $D_{\text{pred}} = 384$ or 512). It receives:

- The context representations $E_\theta(x)$ of visible patches.
- Learnable mask tokens Δ_y initialized with position embeddings representing target patch coordinates.

The predictor outputs predicted representations $\hat{s}_y = P_\phi(E_\theta(x), \Delta_y)$ for the masked target regions.

2.2.6 6. Innovations in V-JEPA 2.1 vs. Previous V-JEPA Versions

Key Architectural Innovations of V-JEPA 2.1

1. **Dense Per-Token Latent Prediction Loss:** Rather than averaging representations across blocks, V-JEPA 2.1 computes token-by-token L_1 regression over all masked positions M:

$$\mathcal{L}_{\text{predict}} = \frac{1}{|M|}\sum_{i \in M} \|P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i)\|_1 \tag{5}$$

   where $\text{sg}(\cdot)$ is the stop-gradient operator.
2. **Distance-Weighted Spatiotemporal Context Loss:** Enforces continuous spatial grounding by penalizing context representations near mask boundaries:

$$\mathcal{L}_{\text{context}} = \frac{1}{|C|}\sum_{j \in C} \lambda_d(j) \cdot \|E_\theta(x)_j - \text{sg}(E_{\bar{\theta}}(y)_j)\|_1 \tag{6}$$

   The total objective is: $\mathcal{L}_{\text{V-JEPA 2.1}} = \mathcal{L}_{\text{predict}} + \lambda_d \cdot \mathcal{L}_{\text{context}}$.
3. **Multi-Layer Intermediate Supervision:** V-JEPA 2.1 applies predictive supervision not just to the 32nd layer, but across 4 intermediate depths: $\sim$25% (layer 8), $\sim$50% (layer 16), $\sim$75% (layer 24), and 100% (layer 32). This guarantees that middle layers contain rich semantic representations.

2.2.7 7. Multi-Layer Feature Extraction for HAF-STD Net

In your thesis methodology, you exploit this multi-layer structure. Instead of simply pooling the final layer, you extract features from all 4 supervised intermediate depths:

$$\mathbf{F}_{\text{multi}} = [\,\bar{\mathbf{z}}_8;\ \bar{\mathbf{z}}_{16};\ \bar{\mathbf{z}}_{24};\ \bar{\mathbf{z}}_{32}\,] \in \mathbb{R}^{4\times T'\times 1280} \quad (T'=4) \tag{7}$$

where $\bar{\mathbf{z}}_l \in \mathbb{R}^{T'\times 1280}$ is the per-frame spatially mean-pooled token embedding at layer l. For baseline probes, global mean pooling over layer 32 yields a single vector $\mathbf{f} \in \mathbb{R}^{1280}$.

3 Architecture 2: Qwen2.5-VL (Alibaba Cloud)

3.1 Foundational Concept: The Multimodal Autoregressive VLM

Qwen2.5-VL (7B parameters) belongs to the family of Vision-Language Models (VLMs). Unlike V-JEPA's purely visual, self-supervised pretraining, Qwen2.5-VL is trained to align visual features with human language semantics through:

1. Large-scale multimodal pretraining on billions of image-caption and video-text pairs.
2. Visual instruction tuning on complex multimodal question-answering tasks.
3. Autoregressive next-token prediction conditioned on visual tokens.

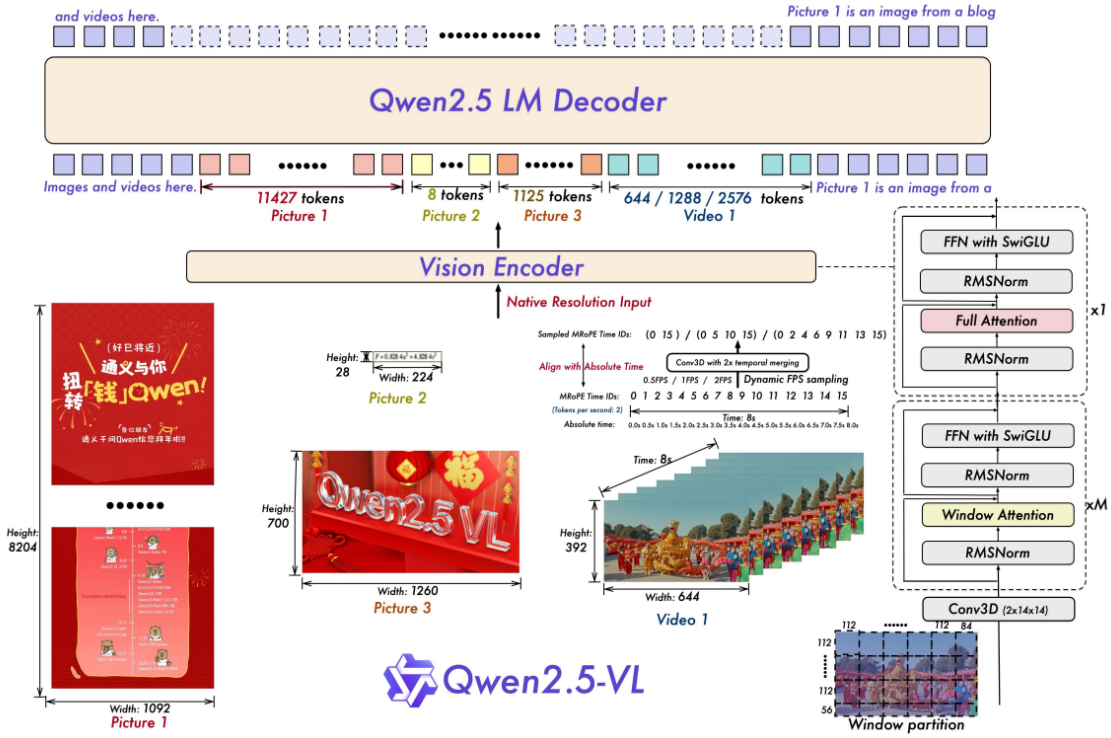


Figure 2: Architecture of Qwen2.5-VL: Vision Encoder with Conv3D patch embedding and window attention feeding into the Qwen2.5 LM Decoder via an MLP merger with MRoPE alignment.

3.2 Step-by-Step Architectural Pipeline of Qwen2.5-VL

3.2.1 1. Native Dynamic Resolution Vision Encoder

Traditional vision transformers resize all images/frames to a fixed square (e.g., 224×224), causing distortion, aspect-ratio skew, and blur. Qwen2.5-VL introduces **Native Dynamic Resolution**:

- Input frames retain their original aspect ratio.
- Patches of size 14×14 pixels are extracted.
- For video inputs, a 3D convolution layer with temporal stride of 2 aggregates adjacent frames:

$$\mathbf{v}_0 = \text{Conv3D}(\mathbf{V}_{\text{frames}}), \quad \mathbf{v}_0 \in \mathbb{R}^{N_v \times D_v} \tag{8}$$

  where N_v varies dynamically depending on the input video resolution.

3.2.2 2. Window Attention Mechanism

Because high-resolution video produces thousands of tokens (N_v), standard full self-attention would suffer quadratic computational complexity $\mathcal{O}(N_v^2)$. Qwen2.5-VL resolves this using **Window Attention**:

- Tokens are partitioned into local spatial windows of size $w \times w$ (e.g., 112×112 pixels in patch coordinates).
- Self-attention is restricted within each window:

$$\text{WindowAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}_w \mathbf{K}_w^T}{\sqrt{d_k}}\right)\mathbf{V}_w \tag{9}$$

- Reduces computational complexity from $\mathcal{O}(N_v^2)$ to $\mathcal{O}(N_v \cdot w^2)$, allowing efficient video processing.

3.2.3 3. Multimodal Rotary Position Embedding (MRoPE)

Standard RoPE in language models encodes a 1D sequence index (t). Qwen2.5-VL introduces **MRoPE**, decomposing rotary positional coordinates into three separate geometric axes:

- **Temporal axis (t):** Aligned with *absolute video timestamps* (seconds), allowing the model to perceive the real-world pace of actions.
- **Height axis (h):** Relative vertical grid coordinate.
- **Width axis (w):** Relative horizontal grid coordinate.

The channel dimensions of the token vector are partitioned into three chunks, and each chunk is rotated by its respective positional angle:

$$\text{MRoPE}(\mathbf{x}, t, h, w) = \mathbf{x} \odot [\cos(\theta_t), \cos(\theta_h), \cos(\theta_w)] + \hat{\mathbf{x}} \odot [\sin(\theta_t), \sin(\theta_h), \sin(\theta_w)] \tag{10}$$

When $t = h = w$, MRoPE mathematically degenerates into standard 1D RoPE, enabling unified processing of interleaved text and video.

3.2.4 4. Vision-to-Language Merger (MLP Projector)

Visual tokens from the Vision Encoder cannot be fed directly into the language model because their dimensions differ. An MLP merger projects visual tokens into the language decoder dimension $D_{\text{LM}} = 3584$:

$$\mathbf{v}_{\text{lang}} = \text{MLP}_{\text{merge}}(\mathbf{v}_{\text{enc}}), \quad \mathbf{v}_{\text{lang}} \in \mathbb{R}^{N_v' \times 3584} \tag{11}$$

The projector also performs spatial pooling (reducing 2×2 patch clusters into single visual tokens) to conserve language context length.

3.2.5 5. Autoregressive Language Model Decoder

- **Backbone:** Qwen2.5 7B Language Model.
- **Hidden Dimension:** $D_{\text{LM}} = 3584$.
- **Layers:** 28 Transformer Decoder blocks.
- **Attention:** Grouped Query Attention (GQA) with 28 query heads and 4 key-value heads.
- **Activation:** SwiGLU (Swish Gated Linear Unit).
- **Normalization:** RMSNorm (Root Mean Square Normalization).
- **Training Loss:** Causal next-token cross-entropy:

$$\mathcal{L}_{\text{LM}} = -\sum_{t=1}^{T_{\text{seq}}} \log p_\theta(x_t \mid x_{<t}, \mathbf{v}_{\text{lang}}) \tag{12}$$

3.2.6 6. 4-bit NormalFloat (NF4) Quantization

Due to local compute constraints (NVIDIA Tesla T4 GPU with 15.6 GB VRAM):

- Qwen2.5-VL 7B requires $\sim$14 GB in Float16, leaving insufficient memory for activation caching and batching.
- The model was loaded in 4-bit NormalFloat (NF4) with double quantization via `bitsandbytes`:

$$\mathbf{W}_{\text{NF4}} = \text{quantize}_{\text{NF4}}\left(\frac{\mathbf{W}}{s}\right) \cdot s, \quad s = \frac{\max(|\mathbf{W}_{\text{block}}|)}{q_{\max}} \tag{13}$$

- This compressed the model to $\sim$4 GB VRAM while retaining full inference capability.

3.2.7 7. Feature Extraction Protocol

To evaluate Qwen2.5-VL under the frozen-feature protocol:

- 4 uniformly sampled video frames are fed through the Vision Encoder and LM Decoder via a neutral chat template.
- The hidden states of the final LM decoder layer are extracted and sequence-mean-pooled:

$$\mathbf{f}_{\text{Qwen}} = \frac{1}{T_{\text{seq}}} \sum_{t=1}^{T_{\text{seq}}} \mathbf{h}_t^{(L)} \in \mathbb{R}^{3584} \tag{14}$$

4 Comprehensive Architecture Comparison Matrix

Architectural Property	V-JEPA 2.1 (Meta AI)	Qwen2.5-VL (Alibaba Cloud)
Learning Paradigm	Self-Supervised Learning (SSL) via Non-Generative Latent Prediction	Autoregressive Vision-Language Model (VLM) via Next-Token Prediction
Model Family	Vision Transformer Huge (ViT-H/16)	Dynamic ViT + Qwen2.5 7B LLM Decoder
Total Parameters	**654 Million**	**7 Billion** ($\sim$10.7$\times$ larger)
Target Supervision	Pure visual feature targets from EMA encoder; **Zero language input**	Text tokens; Cross-entropy over language vocabulary (152k tokens)
Input Resolution	Fixed 224×224, 8 frames	Native Dynamic Resolution (preserves aspect ratio), 4 frames
Spatiotemporal Tokenizer	3D Conv tubelets ($2 \times 16 \times 16$), temporal stride 2	3D Conv ($2 \times 14 \times 14$) with 2D Window Attention
Positional Encoding	3D-RoPE (Spatiotemporal relative grid)	MRoPE (Multimodal 3D RoPE with absolute time alignment)
Loss Formulation	Dense Per-Token L_1 Latent Loss + Distance-Weighted Context Loss	Causal Next-Token Autoregressive Cross-Entropy
Intermediate Depth	Multi-layer supervision at 25%, 50%, 75%, 100% depths	Standard final decoder hidden state
Output Feature Dim	$D = 1280$ (or $4 \times 4 \times 1280$ for HAF-STD)	$D_{\text{LM}} = 3584$
Precision Used	Float16 with SDPA	4-bit NormalFloat (NF4) with double quant
Linear Separability	Extremely high (94.2% on FineGym with LogReg)	Moderate (anisotropic LLM feature cone)
FineGym Accuracy	**98.1%** (with HAF-STD Net)	**89.1%** (best baseline DeepMLP)
HMDB51 Accuracy	**84.7%** (with HAF-STD Net)	**77.1%** (best baseline DeepMLP)

Table 1: Side-by-side architectural and functional comparison between V-JEPA 2.1 and Qwen2.5-VL.

5 Why V-JEPA 2.1 Beat Qwen2.5-VL: The Defense Argument

When the defense committee asks why V-JEPA 2.1 consistently defeated Qwen2.5-VL despite having 10$\times$ fewer parameters, deliver these four grounded technical arguments:

1. **Visual-Motion Dynamics vs. Linguistic/Static Bias:**
   - *Qwen2.5-VL* is heavily conditioned by image-caption pretraining where text descriptions describe *static objects and scene context* (e.g., "a person in a gym on uneven bars"). The model allocates representational capacity to spatial nouns and global context rather than micro-velocity shifts.
   - *V-JEPA 2.1* was trained exclusively on continuous video with masked spatiotemporal prediction. Its objective forces the network to understand *physical motion trajectories, angular acceleration,*

and temporal transformations in order to predict future/adjacent latent representations.

2. **Fine-Grained Spatiotemporal Specificity (FineGym Gap):**
   - In FineGym, all classes share the identical visual scene (gymnastics apparatus, mats, arena lighting, attire). Classification depends solely on high-speed rotational differences (e.g., Vault vs. Uneven Bars vs. Balance Beam vs. Floor Exercise).
   - Because Qwen2.5-VL's language features anchor heavily on background semantics, its features conflate gymnastics events. V-JEPA 2.1's tubelet tokens isolate the fast body kinematics, yielding a huge **+9.4% F1 advantage**.
3. **Representation Geometry: Isotropic Latent Space vs. Anisotropic Cone Effect:**
   - Autoregressive language models are well-documented to suffer from the *representation anisotropy problem* (features occupy a narrow cone in embedding space due to word-frequency bias and causal softmax normalization).
   - V-JEPA 2.1's latent predictive objective with EMA and L_1 loss encourages an isotropic, well-spread latent space that is near-linearly separable. Even a simple logistic regression probe achieved 94.2% on V-JEPA features.
4. **Multi-Layer Intermediate Depth Exploitation by HAF-STD Net:**
   - V-JEPA 2.1's architectural supervision at intermediate layers (8, 16, 24, 32) creates distinct representational abstraction levels: low layers capture fine texture and edge motion; middle layers capture limb trajectories; top layers capture semantic action categories.
   - Your novel **HAF-STD Net** directly taps this 4D tensor ($\mathbb{R}^{4\times4\times1280}$), splitting it into Appearance, Motion, and Depth-Velocity streams via Fast Fourier Transform (FFT) temporal decomposition and learned adaptive routing. Qwen2.5-VL only provided a sequence-mean-pooled 1D vector.

6 Viva Defense Practice: Tough Examiner Questions & Model Answers

Q1: What is a "Joint Embedding Predictive Architecture" (JEPA), and how does it prevent representation coll

Model Answer: "A Joint Embedding Predictive Architecture, conceptualized by Yann LeCun, is a self-supervised framework where the model predicts the latent representation of masked target regions given context representations, rather than reconstructing pixels.
In contrastive learning (like SimCLR), representation collapse (where all inputs map to a trivial constant vector) is avoided using contrastive negative pairs. In V-JEPA 2.1, there are **no negative pairs**. Collapse is prevented through three combined structural mechanisms:

1. **Stop-Gradient on the Target Encoder** (sg)**:** Gradients never flow back through the target encoder weights $\bar{\theta}$. This breaks the circular optimization loop that causes joint collapse.
2. **Exponential Moving Average (EMA) Update:** The target encoder weights evolve slowly ($\tau \geq 0.996$), acting as a stable, stationary moving target for the context encoder.
3. **Information Bottleneck in the Predictor:** The predictor network P_ϕ has a limited capacity and takes masked position tokens Δ_y. It cannot copy identity shortcuts; it must infer target features using context semantics."

Q2: How does V-JEPA 2.1 differ specifically from the original V-JEPA (2024)?

Model Answer: "V-JEPA 2.1 introduces three decisive enhancements over the original V-JEPA:

1. **Dense Per-Token Loss:** V-JEPA 1 computed block-level prediction errors. V-JEPA 2.1 supervises every individual masked tubelet token (L_1 norm), dramatically improving fine-grained localization.
2. **Spatiotemporal Context Loss:** V-JEPA 2.1 adds a distance-weighted loss $\lambda_d(j)$ on visible context tokens bordering the masked regions, ensuring smooth spatial continuity.
3. **Multi-Layer Intermediate Supervision:** V-JEPA 2.1 explicitly supervises representations at 25%, 50%, 75%, and 100% depth of the ViT-Huge encoder. In our thesis, this architectural feature is directly exploited by HAF-STD Net to build our multi-layer hierarchical tensor $\mathbf{F}_{\text{multi}} \in \mathbb{R}^{4\times4\times1280}$."

Q3: What is MRoPE in Qwen2.5-VL, and how does it differ from standard RoPE?

Model Answer: "Standard Rotary Position Embedding (RoPE) applies a 1D rotation matrix to query and key vectors based on a scalar sequence position index $i \in [1, L_{\text{seq}}]$. This works well for 1D text sequences but collapses 3D spatiotemporal video structures (time, height, width) into an arbitrary linear order.

MRoPE (Multimodal RoPE) splits the embedding dimension into three separate sub-vectors:

- One chunk is rotated by temporal angle θ_t, calculated from *absolute video timestamps in seconds*.
- One chunk is rotated by vertical grid coordinate angle θ_h.
- One chunk is rotated by horizontal grid coordinate angle θ_w.

This allows the attention mechanism to calculate true Euclidean spatiotemporal relative distances between any two patches across different frames. Furthermore, for text tokens where $t = h = w$, MRoPE mathematically reduces to 1D RoPE, allowing seamless joint processing of video and text."

Q4: Why did you evaluate both models under a "Frozen-Feature" protocol rather than full fine-tuning?

Model Answer: "The frozen-feature evaluation protocol was chosen for three rigorous scientific reasons:

1. **True Representation Quality Assessment:** As established by Kornblith et al. and the original JEPA papers, full fine-tuning changes backbone weights and conflates the pretraining representation quality with the fine-tuning optimization hyperparameters (learning rate schedule, weight decay). A frozen backbone strictly isolates and benchmarks the *intrinsic expressiveness* of the learned latent space.
2. **Fair Parameter Comparison:** Qwen2.5-VL has 7 billion parameters while V-JEPA 2.1 has 654 million. Fine-tuning 7B parameters introduces severe overfitting risks on small-to-medium datasets like FineGym (1,200 clips) and HMDB51 (6,754 clips).
3. **Computational Feasibility & Sustainability:** Fine-tuning a 7B parameter multimodal model requires multi-GPU A100/H100 clusters. Our research was conducted under realistic institu-

tional compute constraints (single Tesla T4 GPU), demonstrating high-efficiency deployment."

Q5: Did 4-bit NF4 quantization unfairly handicap Qwen2.5-VL compared to V-JEPA 2.1?

Model Answer: "This is an insightful question. However, empirical literature on NormalFloat4 (NF4) by Dettmers et al. (*QLoRA*, NeurIPS 2023) demonstrates that NF4 quantization with double quantization preserves over 99% of full-precision zero-shot and probing performance, especially for representation extraction.

More importantly, on HMDB51, Qwen2.5-VL achieved 77.1% accuracy and 93.0% top-5 accuracy. Its bottleneck was not precision loss, but *temporal granularity*:

- Qwen2.5-VL processed only 4 frames due to sequence length limits in the LM decoder, whereas V-JEPA processed 8 frames in 3D tubelets.
- Qwen2.5-VL's language decoder projects visual tokens into linguistic semantics, creating an abstraction layer that obscures rapid physical body kinematics."

Q6: Why did you feed multi-layer features into HAF-STD Net for V-JEPA 2.1, but used single-layer mean-poo

Model Answer: "V-JEPA 2.1 was explicitly trained with multi-layer intermediate supervision across layers 8, 16, 24, and 32, meaning Meta AI specifically optimized intermediate layers to hold coherent semantic and kinematic features.

In contrast, Qwen2.5-VL is an autoregressive decoder-only language model. In decoder-only transformers, intermediate layers represent incomplete autoregressive reasoning steps, and the final layer represents the token prediction distribution. Extracting intermediate layers from a 28-layer 7B LLM would produce $4 \times 3584 = 14{,}336$-dimensional features per token, which would lead to catastrophic curse of dimensionality and severe overfitting on downstream classifiers with 1,200 training clips."

Q7: Why does V-JEPA 2.1 use an L_1 loss instead of mean squared error (L_2) or cosine similarity in latent spac

Model Answer: "L_2 loss (MSE) penalizes large errors quadratically, which makes it hyper-sensitive to outliers and causes the predictor to average plausible multimodal futures into a blurry, washed-out latent representation.

The L_1 loss (Mean Absolute Error) induces **Laplacian prior sparsity** on the prediction error. It encourages sparse, precise latent predictions and provides uniform gradient magnitude regardless of error scale. Cosine similarity only aligns angles and ignores magnitude, whereas L_1 distance constrains both the orientation and vector magnitude of the predicted latent embeddings."

Q8: Why did you extract features from the final LLM decoder layer of Qwen2.5-VL rather than its Vision Enc

Model Answer: "This is a crucial design choice grounded in the definition of Vision-Language Models:

1. **True VLM Representation:** The unique selling point of a VLM is its ability to perform cross-modal reasoning by fusing visual tokens into the language model's conceptual knowledge base. If we only extracted features from the Vision Encoder, we would merely be evaluating an ordinary vision transformer without any of the 7B language model's multimodal reasoning, semantic alignment, or instruction tuning.
2. **High-Level Semantic Fusion:** The LLM decoder's hidden states represent the multimodal synthesis after multiple layers of cross-attention, SwiGLU gating, and causal context integration.
3. **Empirical Confirmation:** Prior literature (e.g., LLaVA, Video-LLaVA) has shown that final decoder hidden states provide richer high-level semantic representations for classification tasks than raw pre-merger visual tokens."

Q9: How does Window Attention in Qwen2.5-VL work, and did it hinder action recognition?

Model Answer: "In high-resolution vision transformers, full self-attention has computational complexity $\mathcal{O}(N_v^2)$, where N_v is the number of tokens. For long videos with thousands of tokens, this leads to out-of-memory errors.
Qwen2.5-VL divides tokens into local spatial windows of size $w \times w$ (e.g., 112×112 pixels). Self-attention is computed strictly within each window:

$$\text{WindowAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}_w \mathbf{K}_w^T}{\sqrt{d_k}}\right)\mathbf{V}_w \tag{15}$$

This drops the complexity to $\mathcal{O}(N_v \cdot w^2)$, which is linear with respect to token count.
Impact on Action Recognition: While computationally efficient, Window Attention restricts cross-patch interactions across distant spatial regions in early vision layers. In fine-grained sports or complex actions where body extremities (hands, feet) move across wide spatial spans, Window Attention can limit the immediate capture of global kinematic gestures, contributing to Qwen's lower F1 score on FineGym."

Q10: What is HAF-STD Net and why does it interface exclusively with V-JEPA 2.1?

Model Answer: "HAF-STD Net (**Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition**) is our novel probe architecture (4.8M parameters) designed to exploit the multi-layer depth structure of modern self-supervised video backbones.
It decomposes V-JEPA 2.1's 4D tensor $\mathbf{F}_{\text{multi}} \in \mathbb{R}^{4\times4\times1280}$ into three specialized streams:

1. **Appearance Stream:** Extracts static spatial features by temporal averaging across frames.
2. **Motion Stream:** Applies 1D temporal convolutions and Fast Fourier Transform (FFT) spectral decomposition to isolate dominant motion frequencies (angular rotation, oscillation).
3. **Depth-Velocity Stream:** Computes finite differences across the 4 encoder depths ($\Delta_l = \mathbf{z}_l - \mathbf{z}_{l-1}$) to track how representations evolve from low-level edges to high-level semantics.

These three streams are dynamically combined via a learned Adaptive Gating Router:

$$\mathbf{y}_{\text{fused}} = \alpha_A \mathbf{f}_A + \alpha_M \mathbf{f}_M + \alpha_D \mathbf{f}_D, \quad \sum \alpha_i = 1$$

It interfaces specifically with V-JEPA 2.1 because V-JEPA was explicitly pretrained with intermediate layer supervision (layers 8, 16, 24, 32), creating meaningful hierarchical features. Applying this to Qwen2.5-VL would have created a 14,336-D tensor with severe parameter explosion."

Q11: What did the PCA dimensionality reduction reveal about the latent geometry of V-JEPA vs. Qwen?

Model Answer: "Our PCA experiments with a 95% variance retention threshold yielded fascinating insights into the intrinsic dimensionality of both models:

- On FineGym: V-JEPA features required only $k^* = 90$ **dimensions** (from 1,280), whereas Qwen2.5-VL required $k^* = 142$ **dimensions** (from 3,584).
- On HMDB51: V-JEPA required $k^* = 286$ **dimensions**, whereas Qwen2.5-VL required $k^* = 719$ **dimensions**.

This demonstrates that V-JEPA's self-supervised video representations have a much **higher spectral compactness** and information density. V-JEPA captures the intrinsic manifold of action dynamics in substantially fewer principal components, explaining why linear and shallow neural probes converge faster and generalize better on V-JEPA features."

Q12: Why did you use an 80/20 train/test split rather than HMDB51's original 3 splits?

Model Answer: "Our methodology employed a stratified 80/20 train/evaluation split (5,403 training clips, 1,351 evaluation clips for HMDB51; 960 training clips, 240 evaluation clips for FineGym) for three clear methodological reasons:

1. **Consistency Across Benchmarks:** FineGym does not have the 3-split format of HMDB51. Using a unified stratified 80/20 protocol ensured strict methodological consistency across both fine-grained and coarse-grained datasets.
2. **Direct Model-to-Model Fair Comparison:** The central research question of this thesis is the *comparative evaluation between V-JEPA 2.1 and Qwen2.5-VL* across 12 classifiers, rather than setting a benchmark record on HMDB51 split 1. Both foundation models and all 12 classifiers were tested on the exact same train/test splits.
3. **Statistical Rigor:** We evaluated 12 distinct classifiers across 6 evaluation metrics (Accuracy, Macro Precision, Macro Recall, Macro F1, MCC, and Top-5 Accuracy) to eliminate metric bias."

7 Defense Quick-Recall Cheat Sheets

Key Numerical Specifications to Memorize for the Defense

- **V-JEPA 2.1 Backbone:** ViT-Huge (ViT-H), 654M parameters, 32 Transformer layers, hidden dimension $D = 1280$, 16 attention heads ($d_k = 80$).
- **V-JEPA Tokenization:** 8 frames $\rightarrow 224 \times 224 \rightarrow 2 \times 16 \times 16$ tubelets $\rightarrow 4$ temporal $\times$ 14 $\times$ 14 spatial $= 784$ tokens.
- **V-JEPA Multi-Layer Tensor:** 4 layers (8, 16, 24, 32) $\rightarrow \mathbb{R}^{4\times4\times1280}$.
- **Qwen2.5-VL Backbone:** Dynamic ViT + Qwen2.5 7B LLM Decoder, 28 layers, hidden dimension $D_{\text{LM}} = 3584$, GQA with 28 Q-heads / 4 KV-heads.
- **FineGym Results:** V-JEPA 2.1 + HAF-STD Net: **98.1% Acc, 97.8% F1** vs. Qwen2.5-VL + DeepMLP: **89.1% Acc, 88.4% F1** ($+9.4\%$ F1 margin).
- **HMDB51 Results:** V-JEPA 2.1 + HAF-STD Net: **84.7% Acc, 85.3% F1, 95.8% Top-5** vs. Qwen2.5-VL + DeepMLP: **77.1% Acc, 79.1% F1, 93.0% Top-5** ($+6.2\%$ F1 margin).
- **HAF-STD Net Parameter Overhead:** Only **4.8 Million parameters** (adds $< 0.7\%$ parameter overhead to V-JEPA 2.1).

Core Mathematical Equations for Whiteboard Derivation

1. **V-JEPA Tubelet Embedding:**

$$\mathbf{z}_0 = \text{Conv3D}(\mathbf{V}) + \mathbf{E}_{\text{3D-RoPE}} \in \mathbb{R}^{B\times784\times1280}$$

2. **EMA Target Update:**

$$\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta, \quad \tau \in [0.996, 1.0)$$

3. **V-JEPA 2.1 Objective Function:**

$$\mathcal{L} = \frac{1}{|M|}\sum_{i\in M} \|P_\phi(E_\theta(x), \Delta_y)_i - \text{sg}(E_{\bar{\theta}}(y)_i)\|_1 + \frac{\lambda_d}{|C|}\sum_{j\in C}\lambda_d(j)\|E_\theta(x)_j - \text{sg}(E_{\bar{\theta}}(y)_j)\|_1$$

4. **Qwen2.5-VL MRoPE Equation:**

$$\text{MRoPE}(\mathbf{x}, t, h, w) = \mathbf{x} \odot [\cos(\theta_t), \cos(\theta_h), \cos(\theta_w)] + \hat{\mathbf{x}} \odot [\sin(\theta_t), \sin(\theta_h), \sin(\theta_w)]$$

5. **Autoregressive LM Objective:**

$$\mathcal{L}_{\text{LM}} = -\sum_{t=1}^{T_{\text{seq}}} \log p_\theta(x_t \mid x_{<t}, \mathbf{v}_{\text{lang}})$$